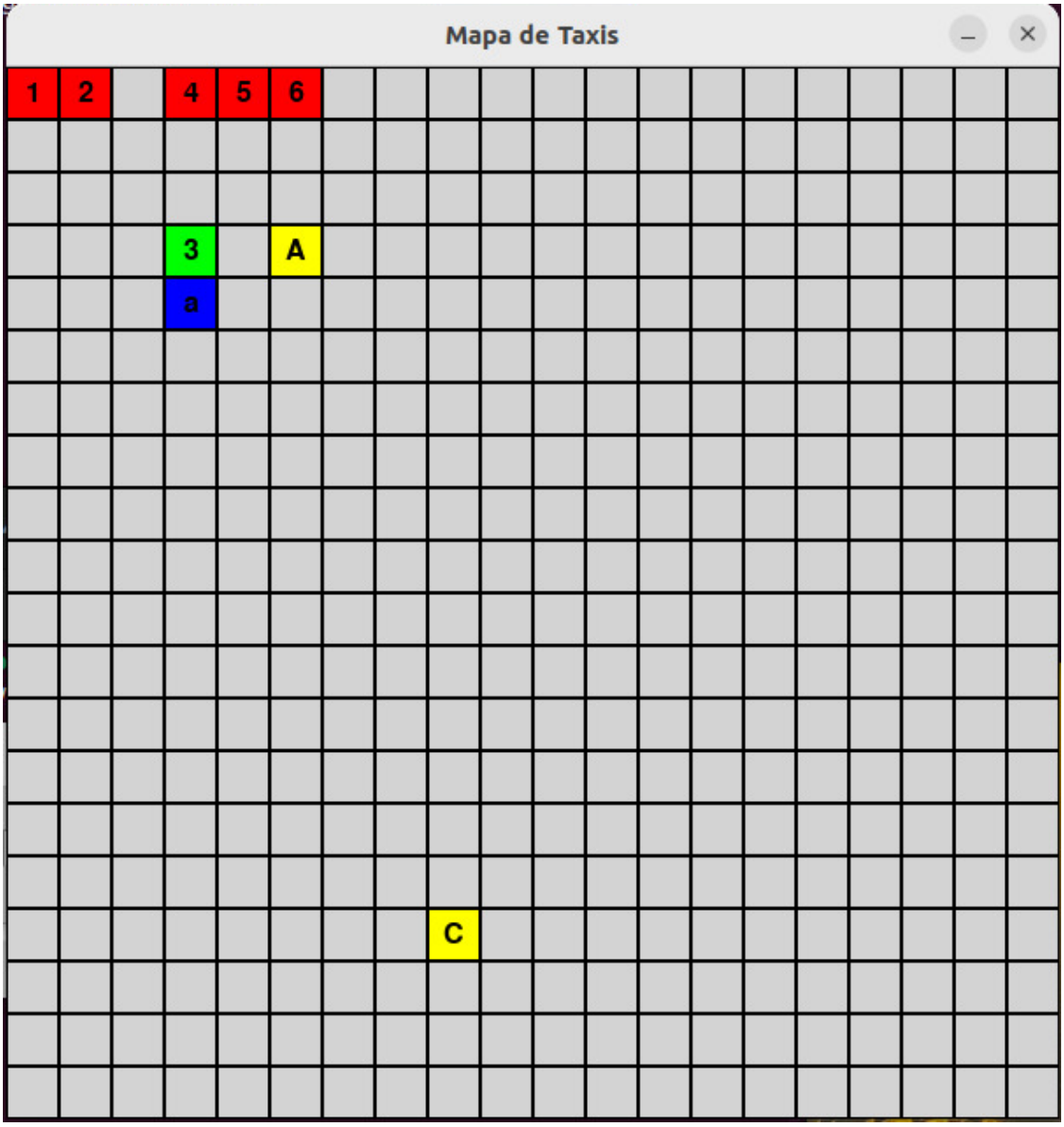
SISTIMAS DISTRIBUIDOS

PRÁCTICA 2
EASY CAB



Aitana Cantó Rico, Levan Kipiani
GRUP0 1 TEO, 02 - 2 PRÁCTICAS

ÍNDICE

COMPONENTES SOFTWARE DESARROLLADOS 2
- **CENTRAL** 2
  - **Interfaz** 2
  - **Detalles Implementación** 3
  - **Conexión Kafka- Sockets** 3
  - **Código** 4
- **DIGITAL ENGINE** 4
  - **Interfaz** 4
  - **Detalles Implementación** 5
  - **Conexión Kafka- Sockets** 5
  - **Código** 5
- **SENSOR** 6
  - **Interfaz** 6
  - **Detalles Implementación** 6
  - **Conexión Sockets** 7
  - **Código** 7
- **CUSTOMER** 7
  - **Interfaz** 7
  - **Detalles Implementación** 8
  - **Conexión Kafka- Sockets** 8
  - **Código** 9
- **MAPA PYGAME** 9
  - **Interfaz** 9
  - **Detalles Implementación** 9
  - **Conexión Kafka- Sockets** 10
  - **Código** 10

GUÍA DE DESPLIEGUE 10

CONTROL DE VERSIONES 13

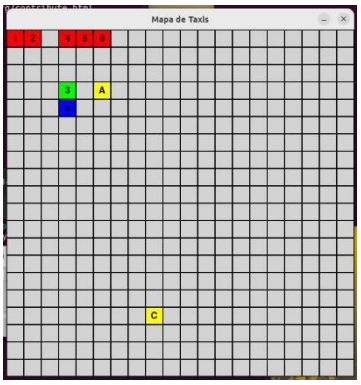
Aitana Cantó Rico
Levan Kipiani

INTRODUCCIÓN

COMPONENTES SOFTWARE DESARROLLADOS

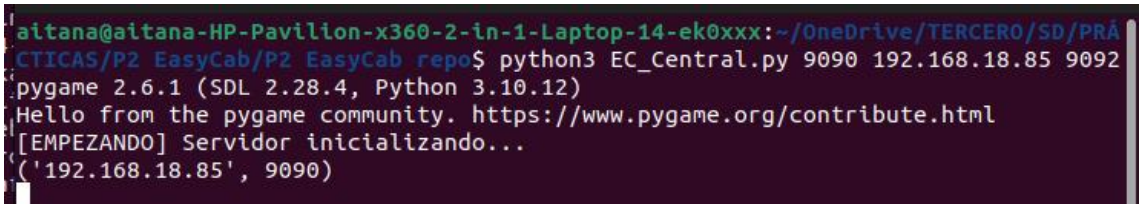
○ CENTRAL

▪ Interfaz



Este archivo es la pieza central del sistema, encargada de coordinar los taxis y las solicitudes de los clientes. Utiliza Kafka para gestionar la comunicación entre los taxis y los clientes, y sockets para recibir las conexiones de los taxis. No interactúa directamente con los usuarios, sino que gestiona la lógica interna del sistema y la actualización del mapa en mapaPygame.

La central no tiene interfaz gráfica de usuario, ya que es un backend que gestiona las solicitudes de los clientes y la comunicación con los taxis y los clientes a través de Kafka y sockets. Su interacción es a través de la línea de comandos, desde donde se reciben los parámetros de configuración. Su principal función es administrar la lógica del sistema central, procesando solicitudes de taxi de clientes y coordinando la comunicación entre clientes, taxis y el sistema de visualización del mapa.



▪ Detalles Implementación

Los principales componentes incluyen:

- **Mapa**: Representado como una matriz de ANCHO x ALTO (20x20) que se va actualizando. La matriz contiene la ubicación de taxis, clientes y destinos, actualizada continuamente. Utiliza un archivo ARCHIVO_MAPA para cargar ubicaciones iniciales y ARCHIVO_TAXI para el estado de cada taxi. Estos archivos almacenan las posiciones y el estado de los taxis, se usan de base de datos. Se garantiza la sincronización con el uso de Lock en operaciones críticas.

  Además, el módulo emplea mapaPygame para visualizar el estado del mapa, actualizando las posiciones de los taxis en tiempo real a medida que se completan las solicitudes.

- **Control de Concurrencia**: Usa hilos para recibir y procesar mensajes de Kafka (conexionKafka) y conexiones de taxis (conexionTaxi). Incluye un Lock para la sincronización de acceso al archivo de taxis y variables compartidas.

- **Control de Peticiones**:

  - **Gestión de Solicitudes de Clientes**: Recibe solicitudes de clientes a través de Kafka. Dependiendo de la disponibilidad de taxis, asigna un taxi a la solicitud o la coloca en una lista de espera.

  - **Asignación de Taxis**: Busca el taxi disponible más cercano y envía las coordenadas de cliente y destino. Si un taxi está ocupado o finaliza un viaje, la central reevalúa la lista de espera.

▪ Conexión Kafka- Sockets

- **Kafka**: Es el medio principal para el intercambio de mensajes entre la central, los clientes y los taxis. Los productores y consumidores de Kafka gestionan las solicitudes de taxi, órdenes de viaje y actualizaciones de mapa:

  - *Topic solicitud_taxi*: Recibe peticiones de los clientes.
  - *Topic procesar_viaje*: Envia órdenes a los taxis, indicando la posición del cliente y destino.
  - *Topic actualizar_mapa*: Envía actualizaciones del mapa a los demás módulos.

- **Sockets**: Los taxis se conectan mediante sockets en startTaxi, donde se valida su ID y se maneja el estado de sus conexiones. Cada taxi tiene un hilo independiente para su verificación y estado. Emplea sockets para establecer la conexión con los taxis, validando su ubicación y estado antes de asignarles un viaje.

▪ Código

Funciones principales:

- meterEnMapa(): Inserta un elemento (cliente o taxi) en el mapa en una posición específica y actualiza la visualización en mapaPygame.
- conexionKafka(): Hilo principal de Kafka, encargado de gestionar las peticiones de clientes y los movimientos de taxis.
- enviarMensajeTaxiOrden(): Envía la orden de viaje a un taxi, incluyendo el punto de recogida y el destino del cliente.
- recibirMensajeTaxi(): Consume mensajes de actualizaciones enviadas por los taxis, reflejando los movimientos en el mapa.
- obtener_lista_ubicaciones(): Lee un archivo JSON que contiene destinos y crea una lista de ubicaciones en un orden específico.
- conexionTaxi(), verificacionTaxi(), y validarUbicacion(): Funciones que gestionan la verificación y comunicación directa con los taxis a través de sockets.
- cambiarEstadoTaxiFichero(), actualizarPosicionTaxiFichero(): Estas funciones actualizan el estado y las posiciones de los taxis en el archivo ARCHIVO_TAXI.

Base de datos de taxis: Usa archivos locales como taxi.txt y mapa.txt para mantener el estado y la ubicación de los taxis.

Hilos: Los hilos de este módulo permiten gestionar múltiples conexiones simultáneamente para que los taxis y clientes puedan interactuar con la central en paralelo.

○ DIGITAL ENGINE

▪ Interfaz

El módulo **EC_DE.py** opera a través de línea de comandos y utiliza argumentos para establecer la configuración de conexión a los otros módulos (IP y puertos). No tiene una interfaz gráfica, ya que su principal función es la de actuar como el "motor digital" para gestionar los viajes de los taxis, comunicándose con otros módulos mediante Kafka y sockets.

```
aitana@aitana-HP-Pavilion-x360-2-in-1-Laptop-14-ek0xxx:~/Descargas/Terminado$ python3 EC_DE.py 192.16
8.18.85 9090 192.168.18.85 9092 192.168.18.85 9091 1
ID asignado con éxito
SERVIDOR CREADO [192.168.18.85 : 9091]
```

▪ Detalles Implementación

- **Variables de Configuración**: El archivo recibe múltiples argumentos para especificar la IP y los puertos del servidor central, el gestor, el sensor, y el identificador del taxi, lo que facilita su despliegue en distintos entornos de red.

- **Sincronización y Bloqueos**: Implementa threading.Event para señalizar la detención del proceso de movimiento del taxi, así como threading.Lock para proteger el acceso concurrente a la variable mapa.

- **Gestión del Viaje**: El taxi sigue un flujo para recoger al cliente, trasladarlo al destino y, al finalizar, notifica al servidor central. La lógica de movimiento permite al taxi moverse hacia el cliente y luego al destino, en ambos casos verificando si el taxi ha alcanzado la posición objetivo. Una vez completado el viaje, el taxi vuelve a estar disponible para nuevos servicios.

▪ Conexión Kafka- Sockets

- **Kafka**:
  - *Topic actualizar_mapa*: Recibe actualizaciones de mapa, reflejando el estado y posiciones de taxis y clientes.
  - *Topic procesar_viaje*: Recibe las órdenes de viaje para el taxi, especificando la posición del cliente y su destino
  - *Topic actualizaciones_taxi*: Envía las posiciones actualizadas del taxi al sistema central.

- **Sockets**:
  - Usa un socket para comunicarse con el sensor (módulo EC_S) para gestionar permisos de movimiento en función de los mensajes recibidos (OK/KO/FIN).

▪ Código

❖ **Movimiento del Taxi**:

La función **realizarViaje()** maneja la secuencia de movimiento del taxi, primero desplazándose hacia el cliente y luego al destino.

La función **actualizarPosicion()** controla el movimiento del taxi hacia el objetivo mediante la función movimiento(), que calcula el desplazamiento basado en el análisis de las coordenadas de destino.

- **Cálculo de Movimiento**:

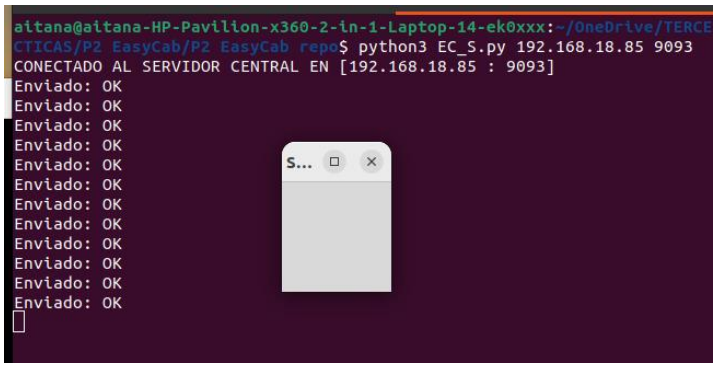
Según la posición actual y las coordenadas de destino, el taxi determina su movimiento en pasos horizontales, verticales o diagonales para acercarse de manera eficiente. Si encuentra obstáculos (una posición ocupada), el taxi intenta realizar movimientos alternativos diagonales o laterales.

- **Otras funciones:**
  - enviarPosicionTaxi(): Envía la posición actual del taxi al sistema central.
  - conexionSensor(): Administra la conexión con el sensor, respondiendo a comandos de movimiento.

SENSOR

Interfaz

Este módulo utiliza una interfaz gráfica simple en Tkinter, que presenta una pequeña ventana para interactuar con el sistema mediante teclas (<s> para pausar o reanudar y <q> para finalizar la conexión con el Digital Engine).



Detalles Implementación

- **Manejo de Eventos**: Configura eventos para sincronizar mensajes entre el hilo principal y el hilo de envío de datos, permitiendo enviar mensajes específicos (OK/KO/FIN) según la tecla presionada.

- **Hilo de Envío de Mensajes**: Un hilo secundario se encarga de enviar mensajes al Digital Engine a intervalos regulares o cuando se produce un cambio en el mensaje (por ejemplo, de OK a KO).

▪ Conexión Sockets

Este módulo se comunica con **EC_DE.py** únicamente a través de un socket TCP. Establece una conexión persistente y envía mensajes de control según el estado del sensor:

- **OK**: Movimiento permitido.
- **KO**: Movimiento detenido.
- **FIN**: Señal para cerrar la conexión y terminar el programa.

▪ Código

Funciones Principales:

- conectarDigitalEngine(): Establece la conexión con **EC_DE.py** mediante un socket TCP.
- enviarMensajes(): Hilo encargado de enviar mensajes periódicos al Digital Engine según el estado actual (OK o KO).
- teclaS_detectada(): Cambia el estado del mensaje entre OK y KO al presionar <s>.
- finalTransmision(): Envía un mensaje FIN y finaliza la conexión cuando se presiona <q>.

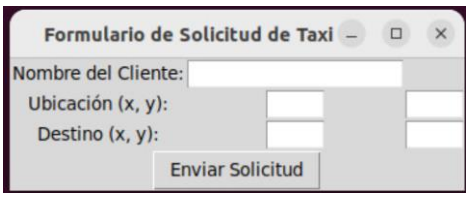
○ CUSTOMER

▪ Interfaz

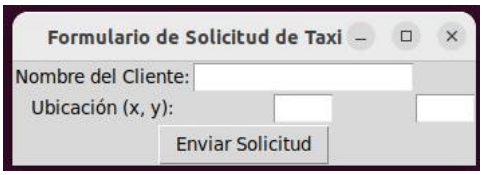
El archivo EC_Customer.py implementa una interfaz gráfica de usuario (GUI) en la que los clientes pueden solicitar un taxi y ver el estado de la solicitud. Para crear la interfaz, utiliza el módulo tkinter de Python, lo que le permite incluir elementos de entrada de texto para que los usuarios introduzcan su nombre, ubicación y destino. La interfaz también proporciona mensajes emergentes (messagebox) que notifican al usuario si su solicitud ha sido procesada, si el taxi está en camino o si ya ha llegado a su destino.

Tiene dos modalidades dependiendo de los comandos que se utilicen:

❖ Comando <-f>:
El cliente pone su nombre, ubicación y destino al que quiere dirigirse.



❖ Comando <nombre.json>:
El cliente escribe en el formulario su nombre y su ubicación. Los destinos a los que se quiere dirigir, se leerán del json pasado por línea de comandos.



▪ Detalles Implementación

La implementación de EC_Customer.py se basa en una arquitectura cliente-servidor, donde los clientes pueden enviar solicitudes de taxi a través de Kafka y recibir mensajes de estado en tiempo real de un servicio central. Los componentes clave incluyen:

1. **Kafka Producer**: Para enviar solicitudes de taxi a un topic de Kafka.
2. **Kafka Consumer**: Para recibir mensajes de estado desde el servicio central.
3. **Manejo de Estados**: Se asigna un identificador de cliente único (id_customer) que permite filtrar los mensajes relevantes para cada cliente. Además, se utiliza un campo codigoMensaje para diferenciar entre tipos de mensajes como confirmaciones, notificaciones de recogida y finalización del viaje.
4. **Recepción de Argumentos:** El módulo espera un parámetro de entrada que puede ser un formulario interactivo (-f) o un archivo JSON con ubicaciones de destino.
5. **Validación de Datos:** Se valida que los campos de ubicación y destino sean válidos y estén dentro de los límites del mapa. Si no, muestra un error.
6. **Función procesar_formulario:** Toma la entrada del usuario, valida los datos y, si son correctos, envía la solicitud a Kafka.
7. **Recepción de Mensajes de Kafka:** Un hilo dedicado escucha mensajes de Kafka para actualizar el estado de la solicitud, como el estado de recogida y finalización.
8. **Funciones auxiliares**: Se emplea json_serializer para serializar mensajes JSON antes de enviarlos, y recibirMensajesDeCentral para gestionar los mensajes entrantes del servidor central.

▪ Conexión Kafka- Sockets

La conexión entre EC_Customer.py y el servicio central es facilitada por Kafka, que actúa como middleware, permitiendo que los clientes publiquen mensajes y reciban actualizaciones de estado a través de topics. En este archivo, no se utilizan sockets directamente; en su lugar, los mensajes se envían y reciben a través de Kafka. La comunicación sigue un flujo asíncrono:

- El cliente envía una solicitud de taxi con **Kafka Producer**.

- El servicio central envía respuestas y actualizaciones de estado usando **Kafka Consumer**, que se configura con un group_id para mantener la sesión de cada

▪ Código

El módulo cuenta con las siguientes funciones principales:

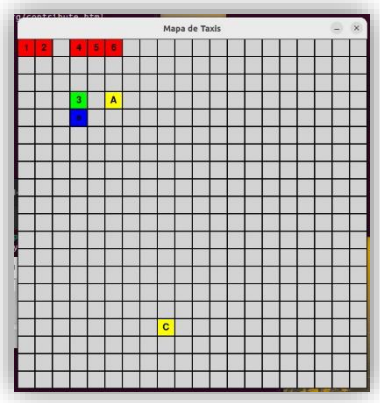
1. **enviar_solicitud_taxi**: Crea un mensaje JSON con la solicitud del cliente y lo envía a Kafka. Envía una solicitud de taxi con los datos ingresados.
2. **recibirMensajesDeCentral**: Escucha actualizaciones del estado de la solicitud y muestra el estado del taxi a través de messagebox. Escucha las respuestas del servidor central y muestra mensajes al usuario.
3. **procesar_formulario**: Controla el flujo de envío de la solicitud al sistema y el inicio de la recepción de mensajes de Kafka. Llama a la función para enviar la solicitud.

Hilos: El módulo lanza un hilo para recibirMensajesDeCentral, permitiendo que la GUI siga activa mientras recibe actualizaciones del servidor.

○ MAPA PYGAME

▪ Interfaz

Este módulo utiliza Pygame para visualizar en tiempo real el estado de un mapa de taxis. Presenta una interfaz gráfica donde los taxis, clientes, y destinos se representan mediante celdas de diferentes colores. Las actualizaciones son continuas y reflejan los cambios en las posiciones de los elementos en el mapa.



▪ Detalles Implementación

- **Visualización en Pygame**: Define el tamaño de cada celda (TAMANO_CELDA), colores para los diferentes elementos (cliente, destino, taxi) y crea una cuadrícula que representa el mapa.
- **Actualización Asíncrona**: Usa una cola (queue.Queue) para manejar actualizaciones desde otros módulos, las cuales se procesan en el bucle principal de Pygame. Esta implementación asegura que los cambios en el mapa sean reflejados sin interferir con el flujo principal del programa.

▪ Conexión Kafka- Sockets

Aunque el módulo no interactúa directamente con Kafka o sockets, se integra en el sistema general permitiendo a los otros módulos enviar actualizaciones de posición a través de la función actualizar_mapa_pygame, que encola las actualizaciones para la visualización en Pygame.

▪ Código

Funciones Principales:

- conectarDigitalEngine(): Establece la conexión con **EC_DE.py** mediante un socket TCP.
- enviarMensajes(): Hilo encargado de enviar mensajes periódicos al Digital Engine según el estado actual (OK o KO).
- teclaS_detectada(): Cambia el estado del mensaje entre OK y KO al presionar <s>.
- finalTransmision(): Envía un mensaje FIN y finaliza la conexión cuando se presiona <q>.
- **iniciar_pygame(ancho, alto)**: Configura la ventana de Pygame en función de las dimensiones del mapa (ancho y alto) y establece una fuente para el texto en las celdas.
- **dibujar_mapa(mapa_data)**: Dibuja el estado actual del mapa. Itera sobre las celdas de mapa_data y asigna un color y texto a cada celda en función de su contenido, diferenciando taxis, clientes y destinos.
- **actualizar_mapa_pygame(mapa_data)**: Recibe el estado del mapa desde otros módulos y lo encola para que main_pygame_loop lo procese.
- **main_pygame_loop()**: Ejecuta el bucle principal de Pygame. Procesa eventos de usuario (como cerrar la ventana) y extrae actualizaciones de la cola para redibujar el mapa.

GUÍA DE DESPLIEGUE

1. KAFKA:

   Descargamos Kafka en nuestro equipo. Será necesario editar los valores listeners y advertisers del archivo server.properties:

   - Listeners: 0.0.0.0
   - Advertisers: Ip del equipo de despliegue.

   Abrimos dos terminales en el directorio de descarga y ejecutamos los siguientes comandos:

- En windows:

  .\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties

  .\bin\windows\kafka-server-start.bat .\config\server.properties

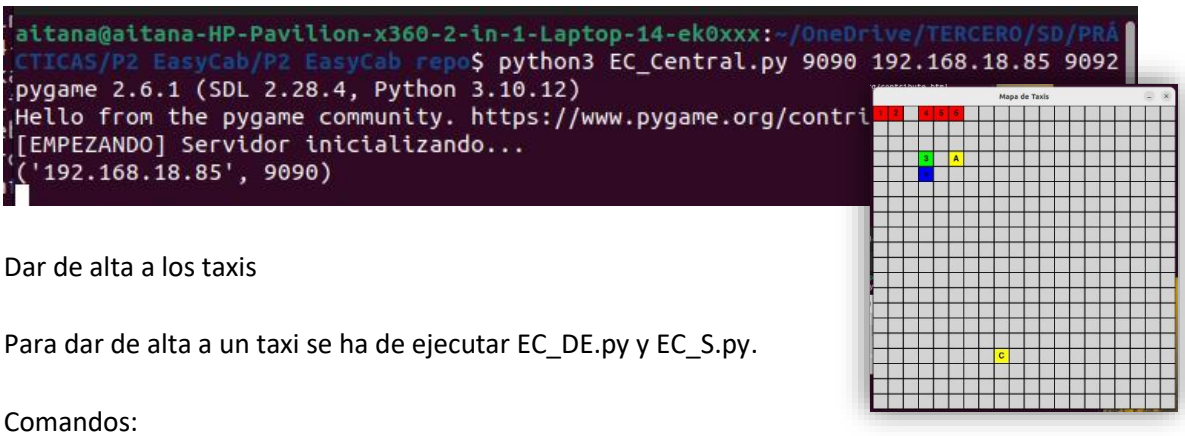
- En Ubuntu:

  $ bin/zookeeper-server-start.sh config/zookeeper.properties

  $ bin/kafka-server-start.sh config/server.properties

2. Desplegar Central:

   Comando: python3 EC_Central.py <Puerto escucha> <IP gestor> <Puerto gestor>

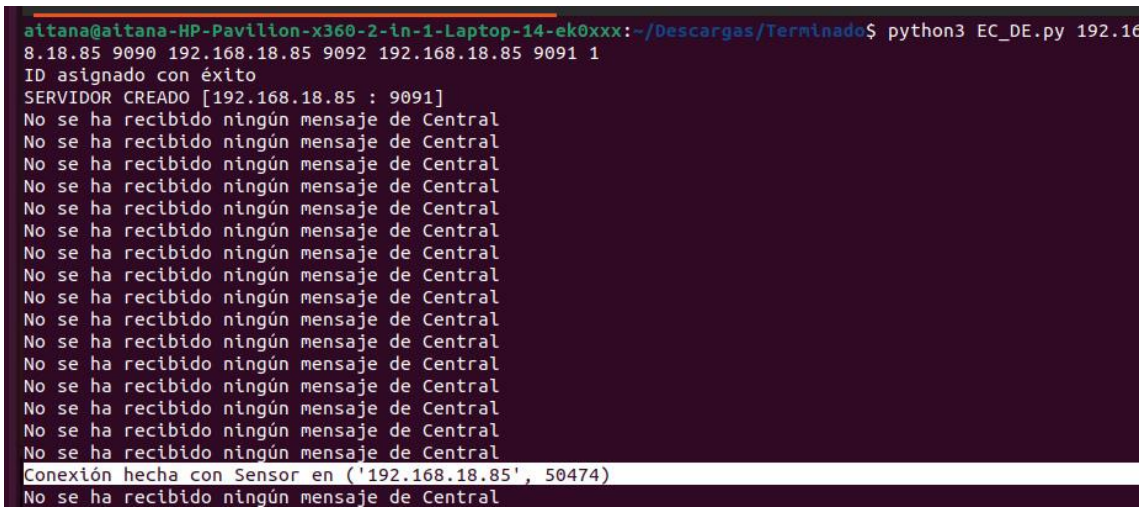


3. Dar de alta a los taxis

   Para dar de alta a un taxi se ha de ejecutar EC_DE.py y EC_S.py.

   Comandos:

   python3 EC_DE.py <IP central> <Puerto central> <IP gestor> <Puerto gestor> <IP Sensor> <Puerto Sensor> <ID Taxi>



   python3 EC_S.py <IP DigitalEngine> <Puerto DigitalEngine>

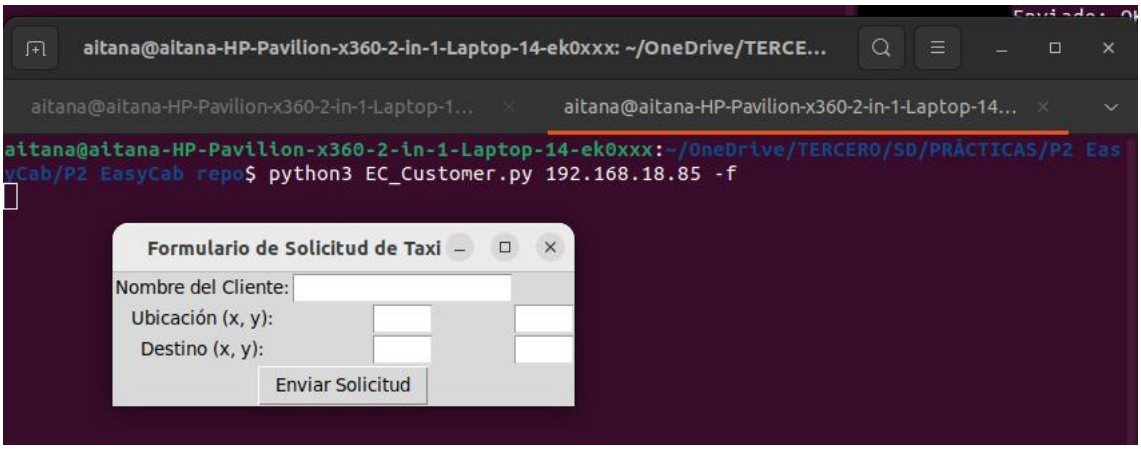
```
aitana@aitana-HP-Pavilion-x360-2-in-1-Laptop-14-ek0xxx:~/Descargas/Terminado$ python3 EC_S.py 192.168
.18.85 9091
CONECTADO AL SERVIDOR CENTRAL EN [192.168.18.85 : 9091]
Enviado: OK
Enviado: OK
```

4. Añadir los customers.

   Tenemos dos opciones de uso:

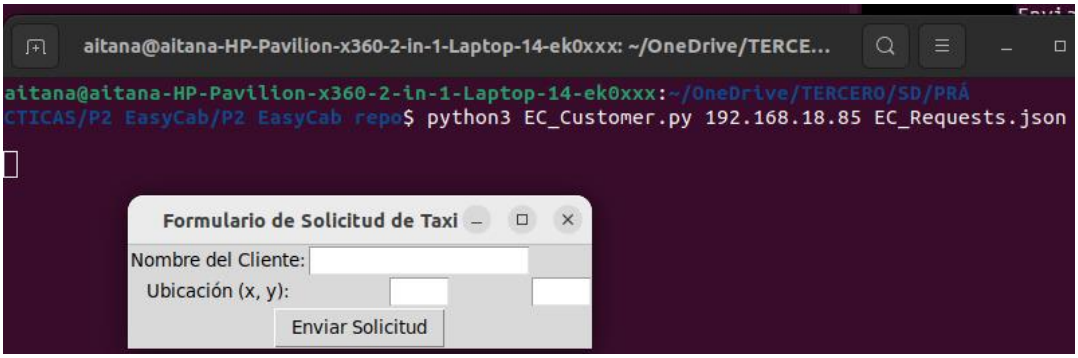
   - **Opción -f:**

     Comando: python3 EC_Customer.py <IP_GESTOR> <-f>



   - **Opción json:**

     Comando: python3 EC_Customer.py <IP_GESTOR> <nombre.json>



CONTROL DE VERSIONES

Para el control de versiones del proyecto, utilizamos Git y GitHub como herramientas principales. Cada miembro trabajó en una rama propia: *Aitana* y *Levan*, donde desarrollaban sus respectivas funcionalidades y realizaban commits individuales. Las ramas se fusionaban en la rama *develop*, que servía para integrar el código y realizar pruebas conjuntas. Una vez que las funcionalidades fueron probadas y aprobadas en *develop*, el código consolidado se fusionó en la rama *master*, la cual se utilizó para almacenar las versiones finales y estables del proyecto.